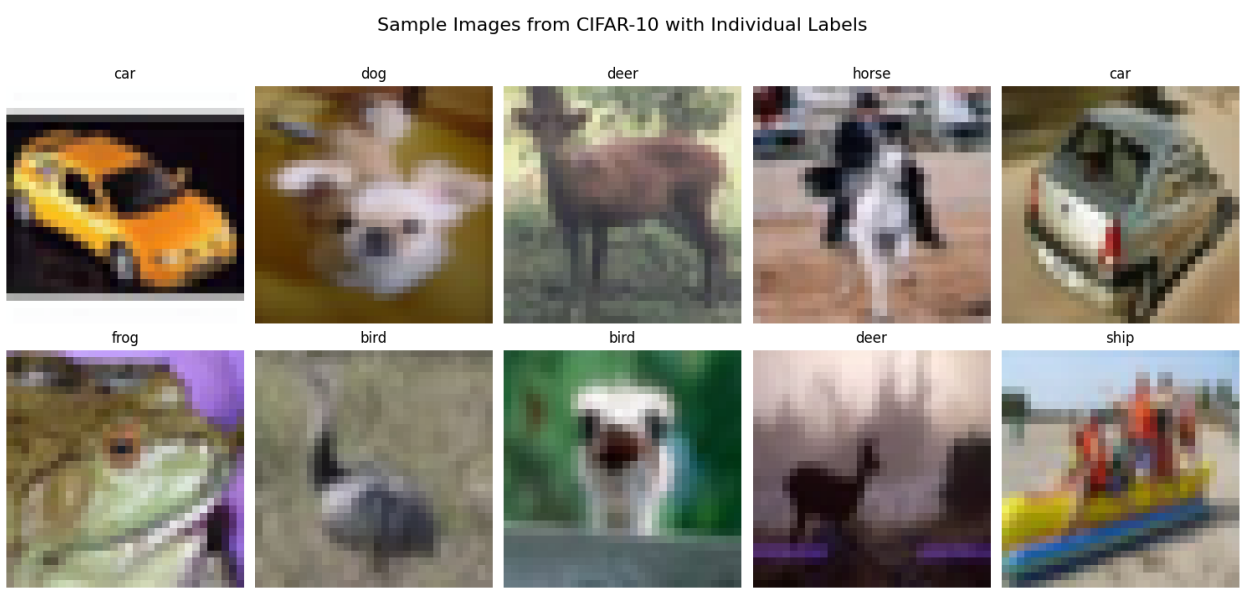
```python
import torch
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np

# Q1. Dataset Exploration

# a) Loading the CIFAR-10 dataset
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.485, 0.456, 0.406), (0.229, 0.224, 0.225))
])

print("Loading CIFAR-10 dataset :- ")
trainset = torchvision.datasets.CIFAR10(root='./data_cifar',
train=True,
                                        download=True,
transform=transform)
testset = torchvision.datasets.CIFAR10(root='./data_cifar',
train=False,
                                       download=True,
transform=transform)

trainloader = torch.utils.data.DataLoader(trainset, batch_size=32,
shuffle=True)
testloader = torch.utils.data.DataLoader(testset, batch_size=32,
shuffle=False)

classes = ('plane', 'car', 'bird', 'cat', 'deer',
           'dog', 'frog', 'horse', 'ship', 'truck')
print("Dataset loaded.")
```

```
Loading CIFAR-10 dataset :-

100%|██████████| 170M/170M [00:03<00:00, 42.8MB/s]

Dataset loaded.
```

```python
# b) Display 10 sample images with their class labels
print("\nDisplaying 10 sample images with individual labels :- ")

dataiter = iter(trainloader)
images, labels = next(dataiter)

plt.figure(figsize=(15, 7)) # Adjusting the size for 2 rows

mean = np.array([0.485, 0.456, 0.406])
std = np.array([0.229, 0.224, 0.225])

# Loop through the first 10 images
```

```
for i in range(10):
    plt.subplot(2, 5, i + 1) # Creating a 2x5 grid

    # Get the image and un-normalize it
    img = images[i]
    npimg = img.numpy()
    npimg = std[:, None, None] * npimg + mean[:, None, None]
    npimg = np.clip(npimg, 0, 1)

    plt.imshow(np.transpose(npimg, (1, 2, 0)))

    # Setting the title to the correct class name
    plt.title(classes[labels[i]])
    plt.axis('off')

plt.suptitle("Sample Images from CIFAR-10 with Individual Labels",
fontsize=16, y=1.02)
plt.tight_layout(pad=1.0)
plt.show()
```

```
Displaying 10 sample images with individual labels :-
```




```
# c) Provide a brief summary of the dataset
print("\nDataset Summary :-")
print(f"Number of training samples: {len(trainset)}")
print(f"Number of testing samples:  {len(testset)}")
sample_image_data, _ = trainset[0]
print(f"Image size (Channels x Height x Width):
{sample_image_data.shape}")
print(f"Number of classes: {len(classes)}")
```

```
Dataset Summary :-
Number of training samples: 50000
Number of testing samples:  10000
Image size (Channels x Height x Width): torch.Size([3, 32, 32])
Number of classes: 10
```

```
import torchvision.transforms as transforms

# Q2. Preprocessing

# a) Normalizing the pixel values appropriately c) data augmentation

transform_train = transforms.Compose([
    transforms.RandomHorizontalFlip(), # Basic augmentation
    transforms.RandomCrop(32, padding=4), # Another common
augmentation
    transforms.ToTensor(),               # Converts to [0,1] and CHW
format
    transforms.Normalize((0.485, 0.456, 0.406), (0.229, 0.224, 0.225))
# Lecture's normalization
])

transform_test = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.485, 0.456, 0.406), (0.229, 0.224, 0.225))
])

# b) Perform one-hot encoding or label transformation as needed.
print("\nb) Label transformation:")
print("   Labels are kept as class indices. PyTorch's
nn.CrossEntropyLoss handles this directly, so no one-hot encoding is
needed.")
```

```
b) Label transformation:
   Labels are kept as class indices. PyTorch's nn.CrossEntropyLoss
handles this directly, so no one-hot encoding is needed.
```

```
import torch.nn as nn
import torch.nn.functional as F  # For ReLU


# Q3. Model Architecture

# a) Define CNN with at least:
#     3 Conv layers
#     1 MaxPool layer
#     1 Dropout layer
#     Fully connected (dense) layer at the end
# c) Use ReLU activation and softmax at the output
```

```python
class CustomCIFAR_CNN(nn.Module):
    def __init__(self):
        super(CustomCIFAR_CNN, self).__init__()

        # Convolutional Layer 1
        self.conv1 = nn.Conv2d(in_channels=3, out_channels=32,
kernel_size=3, padding=1)

        # Convolutional Layer 2
        self.conv2 = nn.Conv2d(in_channels=32, out_channels=64,
kernel_size=3, padding=1)

        # Max Pooling Layer 1
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)

        # Convolutional Layer 3
        self.conv3 = nn.Conv2d(in_channels=64, out_channels=128,
kernel_size=3, padding=1)

        # Dropout Layer 1
        self.dropout1 = nn.Dropout(0.3)

        # This is the hidden dense layer
        self.fc1 = nn.Linear(128 * 16 * 16, 128)

         # Output Layer
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        x = self.pool1(x)

        x = F.relu(self.conv3(x))
        x = self.dropout1(x)

        # Flatten
        x = x.view(-1, 128 * 16 * 16)

        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

# Creating a model instance
cifar_model_q3 = CustomCIFAR_CNN()

# b) Display a model summary
print("\nModel Structure:")
print(cifar_model_q3)
```

```
Model Structure:
CustomCIFAR_CNN(
  (conv1): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
  (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
  (pool1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
  (conv3): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
  (dropout1): Dropout(p=0.3, inplace=False)
  (fc1): Linear(in_features=32768, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=10, bias=True)
)
```

Architectural Choice explanation:

i) 3 Convolutional layers(`conv1 ,conv2 , conv3`) use 3x3 kernels with padding to extract features. The increasing number of filters allows the model to learn progressively more complex patterns from the input image.

ii) 1 MaxPool (`pool1`) after second convolutional layer reduces spatial size and keeps computation efficient.

iii) Dropout (`dropout1`) Layer(with a rate of 0.3) is applied after the third convolutional layer's activation which helps reduce overfitting.

iv) `fc1` is a hidden dense layer (128 units) that takes the flattened output from the convolutional blocks. `fc2` is the final output dense layer (10 units), corresponding to the 10 CIFAR-10 classes.

v) It uses ReLU activations which introduces non-linearity, allowing the model to learn complex relationships And outputs raw logits (will apply softmax during loss calculation later).

```
# Q4. Training (PyTorch) - Concise Loop


# Assumes 'cifar_model_q3' as we have used earlier in the above
questions.
#'trainloader', and 'testloader' are defined above.
import torch.optim as optim
from tqdm.auto import tqdm # For progress bars
import time
import matplotlib.pyplot as plt


# Q4: Training
```

```python
print("\n Q4: Training ")
model = cifar_model_q3
# a) Compiling the model using an appropriate optimizer and loss
function.
device = torch.device('cuda' if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
print("Model compiled (device, loss, optimizer set).")
```

```
 Q4: Training
Using device: cuda
Model compiled (device, loss, optimizer set).
```

```python
# b) Train for at least 10 epochs with training and validation
accuracy plotted.


num_epochs = 10
history = {'train_loss': [], 'train_acc': [], 'val_loss': [],
'val_acc': []}

print(f"\nStarting training for {num_epochs} epochs :")
for epoch in range(num_epochs):
    epoch_start_time = time.time()
    print(f"\n   Epoch {epoch + 1}/{num_epochs} :- ")

    # Training phase :-
    model.train()
    running_train_loss = 0.0
    correct_train = 0
    total_train = 0
    for images, labels in tqdm(trainloader, desc="Training"):
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_train_loss += loss.item() * images.size(0)
        _, predicted_train = torch.max(outputs.data, 1)
        total_train += labels.size(0)
        correct_train += (predicted_train == labels).sum().item()

    epoch_train_loss = running_train_loss / total_train
    epoch_train_acc = 100. * correct_train / total_train
```

```
    history['train_loss'].append(epoch_train_loss)
    history['train_acc'].append(epoch_train_acc)

    # Validation phase :-
    model.eval()
    running_val_loss = 0.0
    correct_val = 0
    total_val = 0
    with torch.no_grad():
        for images, labels in tqdm(testloader, desc="Validation"):
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)

            running_val_loss += loss.item() * images.size(0)
            _, predicted_val = torch.max(outputs.data, 1)
            total_val += labels.size(0)
            correct_val += (predicted_val == labels).sum().item()

    epoch_val_loss = running_val_loss / total_val
    epoch_val_acc = 100. * correct_val / total_val
    history['val_loss'].append(epoch_val_loss)
    history['val_acc'].append(epoch_val_acc)

    epoch_end_time = time.time()
    print(f"Epoch {epoch + 1} Summary:")
    print(f"  Train Loss: {epoch_train_loss:.4f}, Train Acc:
{epoch_train_acc:.2f}%")
    print(f"  Val Loss:   {epoch_val_loss:.4f}, Val Acc:
{epoch_val_acc:.2f}%")
    print(f"  Time: {epoch_end_time - epoch_start_time:.2f}s")

print("\nTraining finished.")
```

```
Starting training for 10 epochs :

   Epoch 1/10 :-
```

{"model_id":"1b7862b206654ee1b8acf260573da957","version_major":2,"version_minor":0}

{"model_id":"10f8c46a7ae5426dbf49208d2385852a","version_major":2,"version_minor":0}

```
Epoch 1 Summary:
  Train Loss: 1.1819, Train Acc: 58.05%
  Val Loss:   0.9179, Val Acc:   67.87%
  Time: 26.85s

   Epoch 2/10 :-
```

{"model_id":"5cf628f3f8fb45c7a9cce05b1f51aff8","version_major":2,"version_minor":0}

{"model_id":"23a5a4e05aab439ba3e7dfb5c664b901","version_major":2,"version_minor":0}

```
Epoch 2 Summary:
  Train Loss: 0.7741, Train Acc: 72.89%
  Val Loss:   0.7752, Val Acc:   72.43%
  Time: 24.76s

   Epoch 3/10 :-
```

{"model_id":"7a47fd2279d145bfa3403394beaf2437","version_major":2,"version_minor":0}

{"model_id":"51e7f9f78d124a739842f4d6c4259ea6","version_major":2,"version_minor":0}

```
Epoch 3 Summary:
  Train Loss: 0.5813, Train Acc: 79.56%
  Val Loss:   0.8123, Val Acc:   72.47%
  Time: 25.45s

   Epoch 4/10 :-
```

{"model_id":"c973273c011f4a6980cb277125d65429","version_major":2,"version_minor":0}

{"model_id":"866e67626c8d4ff8bd5bfd8ac39f4ae2","version_major":2,"version_minor":0}

```
Epoch 4 Summary:
  Train Loss: 0.4189, Train Acc: 85.08%
  Val Loss:   0.8473, Val Acc:   74.59%
  Time: 24.04s

   Epoch 5/10 :-
```

{"model_id":"61fd0ee79dac412289bf3ce398fdcb35","version_major":2,"version_minor":0}

{"model_id":"b8c5e3a9cf454f1488396ef825bf0056","version_major":2,"version_minor":0}

```
Epoch 5 Summary:
  Train Loss: 0.3028, Train Acc: 89.25%
  Val Loss:   0.9409, Val Acc:   73.90%
  Time: 24.06s

   Epoch 6/10 :-
```

{"model_id":"0eb25ea947c94e59a4a80bf033d54e93","version_major":2,"version_minor":0}

{"model_id":"41026bc876604ddbb132f2e6c8175da6","version_major":2,"version_minor":0}

```
Epoch 6 Summary:
  Train Loss: 0.2305, Train Acc: 91.90%
  Val Loss:   1.0655, Val Acc:   73.57%
  Time: 24.40s

   Epoch 7/10 :-
```

{"model_id":"88a5a3a1ca154e659c7d0802721d9dad","version_major":2,"version_minor":0}

{"model_id":"b0980e9ffb7246beb96d81c9817f52e8","version_major":2,"version_minor":0}

```
Epoch 7 Summary:
  Train Loss: 0.1831, Train Acc: 93.60%
  Val Loss:   1.1627, Val Acc:   73.20%
  Time: 24.43s

   Epoch 8/10 :-
```

{"model_id":"019b3563568b4f748bb8f2c346c356f3","version_major":2,"version_minor":0}

{"model_id":"16bd57439c3e42b4bfd70a40b4b035f5","version_major":2,"version_minor":0}

```
Epoch 8 Summary:
  Train Loss: 0.1583, Train Acc: 94.61%
  Val Loss:   1.3343, Val Acc:   73.11%
  Time: 24.35s

   Epoch 9/10 :-
```

{"model_id":"3bfed95f2f5e44d09ac91a73474361ed","version_major":2,"version_minor":0}

{"model_id":"4809f4a3b7284bcca008a1bd2e1963cf","version_major":2,"version_minor":0}

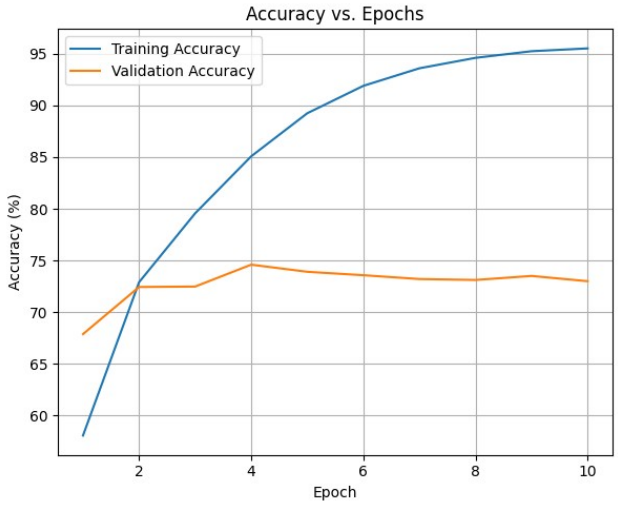
```
Epoch 9 Summary:
  Train Loss: 0.1422, Train Acc: 95.25%
  Val Loss:   1.2585, Val Acc:   73.50%
  Time: 24.39s

   Epoch 10/10 :-
```

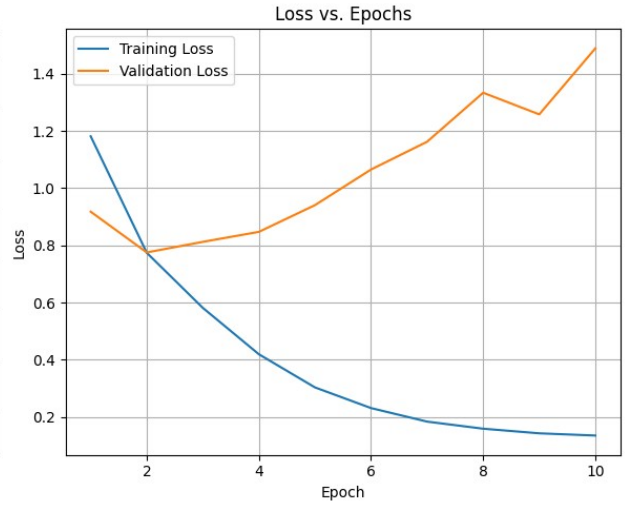
{"model_id":"4af3a30ff1ae40e78fed96a090096ba1","version_major":2,"version_minor":0}

{"model_id":"7fd50e30275749589b90a8ac5a72ec2b","version_major":2,"version_minor":0}

```
Epoch 10 Summary:
  Train Loss: 0.1344, Train Acc: 95.52%
  Val Loss:   1.4894, Val Acc:   72.99%
  Time: 24.41s

Training finished.
```

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(range(1, num_epochs + 1), history['train_acc'],
label='Training Accuracy')
plt.plot(range(1, num_epochs + 1), history['val_acc'],
label='Validation Accuracy')
plt.title('Accuracy vs. Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy (%)')
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(range(1, num_epochs + 1), history['train_loss'],
label='Training Loss')
plt.plot(range(1, num_epochs + 1), history['val_loss'],
label='Validation Loss')
plt.title('Loss vs. Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```





Insights from Training

- **Learning Curve Behavior:**
  - **Training Set:** Accuracy jumps from 58% to 95%. Loss drops from 1.18 to 0.13 by Epoch 10.
  - **Validation Set:** Accuracy goes up to 74% by Epoch 4, then stays around 73%. Loss dips to 0.78 at Epoch 2, then goes up to 1.49.
- **Convergence**
  - The model clearly converges on the training data, achieving high accuracy and low loss.
  - The model gets really good on the training data by Epoch 4.
  - But for new data (validation), it stops getting better after Epoch 2–4.
- **Signs of Overfitting**
  - By Epoch 10, training accuracy is 95% but validation is just 73%.
  - Training loss keeps falling, but validation loss starts climbing after Epoch 2.

```
import seaborn as sns
from sklearn.metrics import classification_report, confusion_matrix

# Q5. Evaluation


model.to(device)


# a) Evaluate the trained model on the test set.
model.eval() # Setting the model to evaluation mode

all_labels = []
all_predictions = []
```

```
running_test_loss = 0.0
correct_test = 0
total_test = 0



print("Evaluating on the test set :- ")
with torch.no_grad():
    for batch_idx, (images, labels) in enumerate(testloader):
        images, labels = images.to(device), labels.to(device)

        outputs = model(images)
        loss = criterion(outputs, labels)
        running_test_loss += loss.item() * images.size(0)

        _, predicted = torch.max(outputs.data, 1)

        total_test += labels.size(0)
        correct_test += (predicted == labels).sum().item()

        all_labels.extend(labels.cpu().numpy())
        all_predictions.extend(predicted.cpu().numpy())

        # Displaying the progress for a simple feedback
        if (batch_idx + 1) % 50 == 0:
            print(f"  Processed batch {batch_idx +
1}/{len(testloader)}")
```

```
Evaluating on the test set :-
  Processed batch 50/313
  Processed batch 100/313
  Processed batch 150/313
  Processed batch 200/313
  Processed batch 250/313
  Processed batch 300/313
```

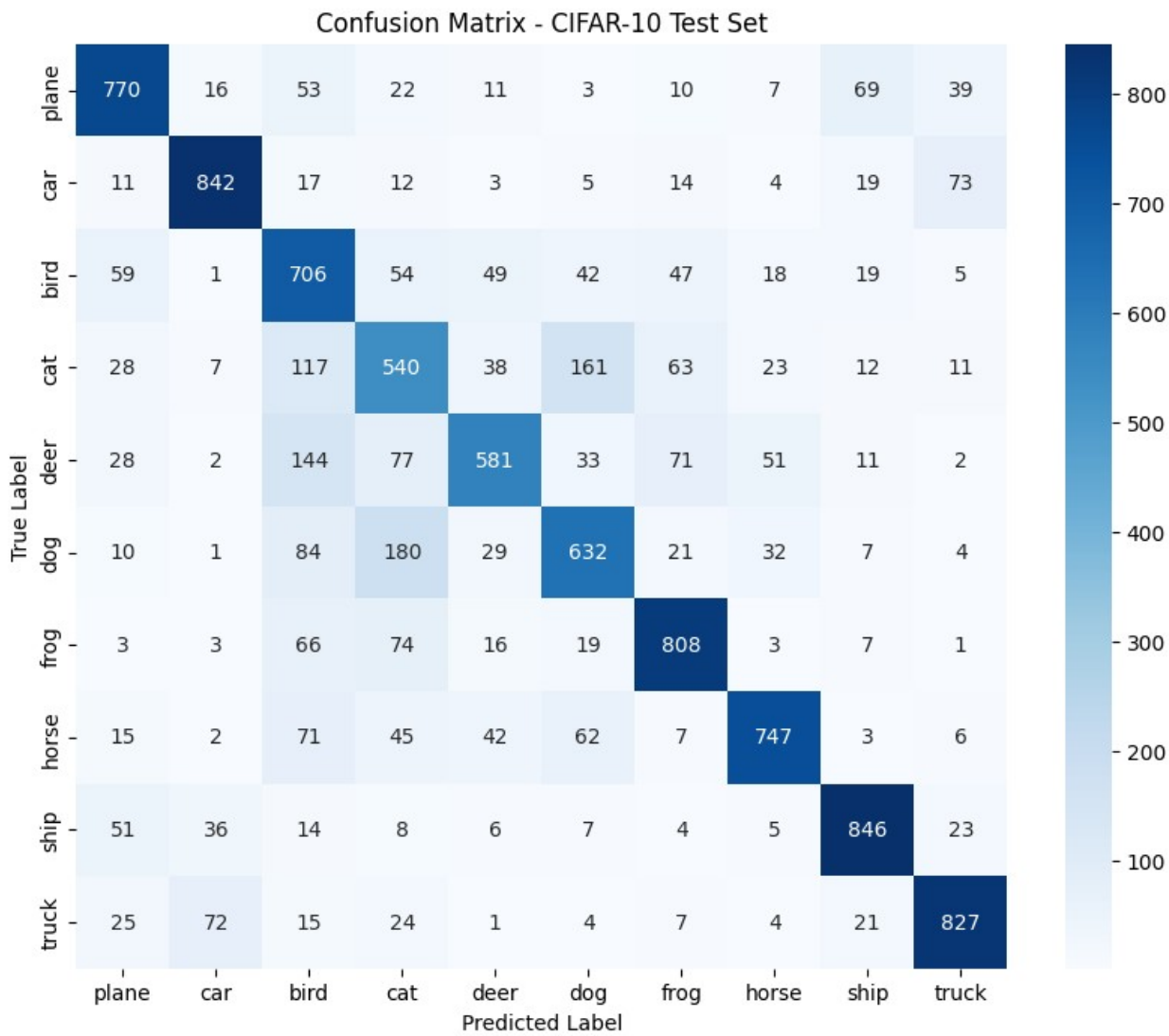
```
avg_test_loss = running_test_loss / total_test
test_accuracy = 100. * correct_test / total_test

print(f"\nTest Set Performance:")
print(f"  Average Loss: {avg_test_loss:.4f}")
print(f"  Accuracy: {test_accuracy:.2f}%")
```

```
Test Set Performance:
  Average Loss: 1.4894
  Accuracy: 72.99%
```

```
# b) confusion matrix
print("\nGenerating Confusion Matrix...")
cm = confusion_matrix(all_labels, all_predictions)
```

```
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=classes, yticklabels=classes)
plt.title('Confusion Matrix - CIFAR-10 Test Set')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

```
Generating Confusion Matrix...
```




```
# c) Compute and display accuracy, precision, recall, and F1-score
print("\nClassification Report:")
report = classification_report(np.array(all_labels),
np.array(all_predictions),
```

```
                                target_names=classes, digits=3)
print(report)
```

```
Classification Report:
              precision    recall  f1-score   support

       plane      0.770     0.770     0.770      1000
         car      0.857     0.842     0.850      1000
        bird      0.549     0.706     0.617      1000
         cat      0.521     0.540     0.530      1000
        deer      0.749     0.581     0.654      1000
         dog      0.653     0.632     0.642      1000
        frog      0.768     0.808     0.788      1000
       horse      0.836     0.747     0.789      1000
        ship      0.834     0.846     0.840      1000
       truck      0.835     0.827     0.831      1000

    accuracy                           0.730     10000
   macro avg      0.737     0.730     0.731     10000
weighted avg      0.737     0.730     0.731     10000
```

```
device = torch.device('cuda' if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()
print(f"Using device: {device}")
```

```
Using device: cuda
```

```
import copy



# 1. Implement Early Stopping (and Model Checkpointing for best model)



print("\nImplementing Training with Early Stopping")

# Early stopping parameters
patience = 3 # Number of epochs to wait for improvement before
stopping
best_val_loss = float('inf')
best_model_state = None
epochs_no_improve = 0
num_epochs_early_stopping = 20 # Train for more epochs to see early
stopping potentially trigger

# Instantiate a new model for this part
model_for_early_stopping = CustomCIFAR_CNN().to(device)
optimizer_es = optim.Adam(model_for_early_stopping.parameters(),
```

```
lr=0.001)

train_losses_es, val_losses_es = [], []
train_accs_es, val_accs_es = [], []
```

Implementing Training with Early Stopping

```
for epoch in range(num_epochs_early_stopping):
    # Training Phase :-
    model_for_early_stopping.train()
    running_train_loss = 0.0
    correct_train, total_train = 0, 0
    for images, labels in tqdm(trainloader, desc=f"ES Train
E{epoch+1}"):
        images, labels = images.to(device), labels.to(device)
        optimizer_es.zero_grad()
        outputs = model_for_early_stopping(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer_es.step()
        running_train_loss += loss.item() * images.size(0)
        _, predicted = torch.max(outputs.data, 1)
        total_train += labels.size(0)
        correct_train += (predicted == labels).sum().item()

    epoch_train_loss = running_train_loss / total_train
    epoch_train_acc = 100. * correct_train / total_train
    train_losses_es.append(epoch_train_loss)
    train_accs_es.append(epoch_train_acc)

    # Validation Phase
    model_for_early_stopping.eval()
    running_val_loss = 0.0
    correct_val, total_val = 0, 0
    with torch.no_grad():
        for images, labels in tqdm(testloader, desc=f"ES Val
E{epoch+1}"):
            images, labels = images.to(device), labels.to(device)
            outputs = model_for_early_stopping(images)
            loss = criterion(outputs, labels)
            running_val_loss += loss.item() * images.size(0)
            _, predicted = torch.max(outputs.data, 1)
            total_val += labels.size(0)
            correct_val += (predicted == labels).sum().item()

    epoch_val_loss = running_val_loss / total_val
    epoch_val_acc = 100. * correct_val / total_val
    val_losses_es.append(epoch_val_loss)
```

```
    val_accs_es.append(epoch_val_acc)

    print(f"Epoch {epoch+1}/{num_epochs_early_stopping}: Train Loss: 
{epoch_train_loss:.4f}, Train Acc: {epoch_train_acc:.2f}% | Val Loss: 
{epoch_val_loss:.4f}, Val Acc: {epoch_val_acc:.2f}%")

    # Early stopping check
    if epoch_val_loss < best_val_loss:
        best_val_loss = epoch_val_loss
        best_model_state = 
copy.deepcopy(model_for_early_stopping.state_dict()) # Save best model 
weights
        epochs_no_improve = 0
        print(f"  Validation loss improved to {best_val_loss:.4f}. 
Saving model state.")
    else:
        epochs_no_improve += 1
        print(f"  No improvement in validation loss for 
{epochs_no_improve} epoch(s).")
        if epochs_no_improve >= patience:
            print(f"Early stopping triggered after {epoch+1} epochs.")
            break
```

{"model_id":"bce8cb9288c94779a4bbb884a1cb0e3b","version_major":2,"version_minor":0}

{"model_id":"2e8278587df54d6d96001dec398672f3","version_major":2,"version_minor":0}

```
Epoch 1/20: Train Loss: 1.2116, Train Acc: 56.40% | Val Loss: 0.9387, 
Val Acc: 66.63%
  Validation loss improved to 0.9387. Saving model state.
```

{"model_id":"3d9655f482f44c6eb7a2c357b0fbfec8","version_major":2,"version_minor":0}

{"model_id":"bee48d24d17340e2b9f82f2b24610aa0","version_major":2,"version_minor":0}

```
Epoch 2/20: Train Loss: 0.8181, Train Acc: 71.16% | Val Loss: 0.8503, 
Val Acc: 69.79%
  Validation loss improved to 0.8503. Saving model state.
```

{"model_id":"cf5606f182be48cc9ee26450dad60e90","version_major":2,"version_minor":0}

{"model_id":"0859fb7b04284a97ac406df7425a66f9","version_major":2,"version_minor":0}

```
Epoch 3/20: Train Loss: 0.6432, Train Acc: 77.41% | Val Loss: 0.7954,
Val Acc: 73.18%
  Validation loss improved to 0.7954. Saving model state.
```

{"model_id":"5267264ed0f6495690165c0c92d813f8","version_major":2,"version_minor":0}

{"model_id":"5ed408f3ddd34f03b39317c206e8de6e","version_major":2,"version_minor":0}

```
Epoch 4/20: Train Loss: 0.4999, Train Acc: 82.27% | Val Loss: 0.7974,
Val Acc: 73.78%
  No improvement in validation loss for 1 epoch(s).
```

{"model_id":"644ec88d39254d7a9f0255d96087bf31","version_major":2,"version_minor":0}

{"model_id":"ef9e4dd6577c497e9b47f6db930fba9f","version_major":2,"version_minor":0}

```
Epoch 5/20: Train Loss: 0.3764, Train Acc: 86.77% | Val Loss: 0.8464,
Val Acc: 73.85%
  No improvement in validation loss for 2 epoch(s).
```

{"model_id":"acb673d3f41446ab8e6c63a509aed130","version_major":2,"version_minor":0}

{"model_id":"80bc3f66f4c8415eaba06a886cf8a6a2","version_major":2,"version_minor":0}

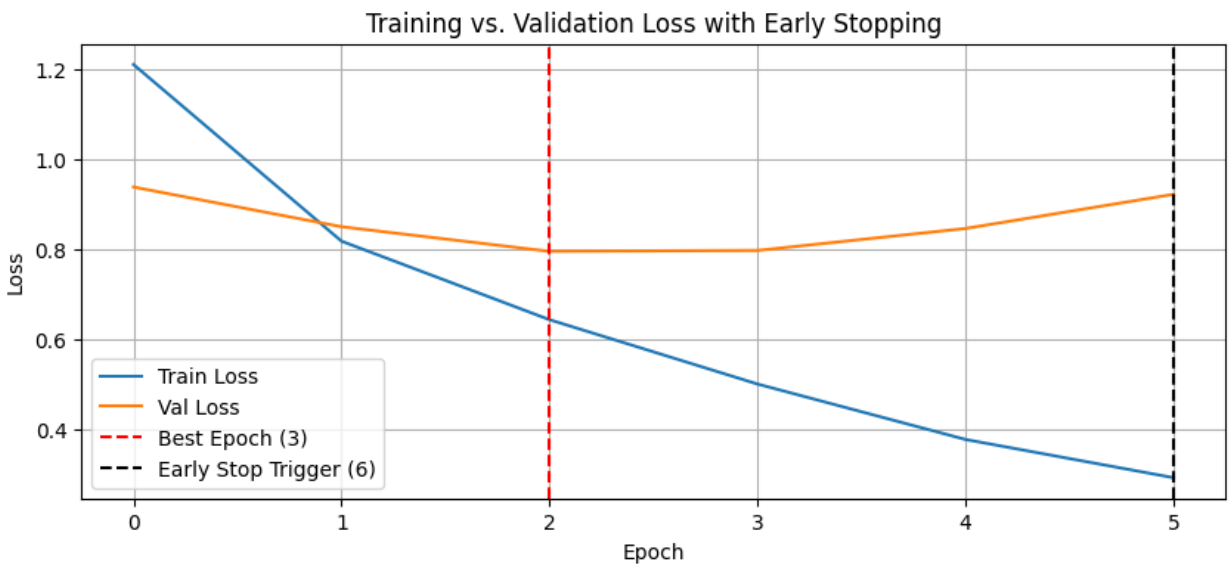
```
Epoch 6/20: Train Loss: 0.2913, Train Acc: 89.81% | Val Loss: 0.9225,
Val Acc: 74.71%
  No improvement in validation loss for 3 epoch(s).
Early stopping triggered after 6 epochs.
```

```
# Loading the best model weights if early stopping happened or
training finished
if best_model_state:
    print("Loading best model weights achieved during early
stopping.")
    model_for_early_stopping.load_state_dict(best_model_state)
else:
    print("Training finished without early stopping or no improvement
found to save a best model.")
```

```
Loading best model weights achieved during early stopping.
```

```
# Plotting training and validation loss with early stopping
plt.figure(figsize=(10, 4))
plt.plot(train_losses_es, label='Train Loss')
plt.plot(val_losses_es, label='Val Loss')
if epochs_no_improve >= patience and len(val_losses_es) <
```

```
num_epochs_early_stopping : # If early stopping triggered
    plt.axvline(len(train_losses_es) -1 - patience, color='r',
linestyle='--', label=f'Best Epoch ({len(train_losses_es)-patience})')
    plt.axvline(len(train_losses_es) -1 , color='k', linestyle='--',
label=f'Early Stop Trigger ({len(train_losses_es)})')

plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training vs. Validation Loss with Early Stopping')
plt.legend()
plt.grid(True)
plt.show()
```




```
# 2. Try Tuning Hyperparameters (Example: Learning Rate)
print("\nTrying Hyperparameter Tuning (Learning Rate)...")
learning_rates_to_try = [0.0005, 0.001, 0.005]
lr_results = {}
num_epochs_tuning = 5 # Short training for hyperparameter demo

for lr_val in learning_rates_to_try:
    print(f"\n  Tuning with Learning Rate: {lr_val}")
    # Create a new model instance for each learning rate
    temp_model = CustomCIFAR_CNN().to(device)
    temp_optimizer = optim.Adam(temp_model.parameters(), lr=lr_val)

    for epoch_tune in range(num_epochs_tuning):
        temp_model.train()
        for images, labels in tqdm(trainloader, desc=f"Tune
LR={lr_val} E{epoch_tune+1}", leave=False):
            images, labels = images.to(device), labels.to(device)
            temp_optimizer.zero_grad()
```

```
            outputs = temp_model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            temp_optimizer.step()
        # Simple print for tuning progress
        if (epoch_tune + 1) % num_epochs_tuning == 0:
             print(f"    Finished {num_epochs_tuning} epochs for
LR={lr_val}")
    # Evaluate this temp model on validation set
    temp_model.eval()
    correct_val_tune, total_val_tune = 0, 0
    with torch.no_grad():
        for images, labels in testloader:
            images, labels = images.to(device), labels.to(device)
            outputs = temp_model(images)
            _, predicted = torch.max(outputs.data, 1)
            total_val_tune += labels.size(0)
            correct_val_tune += (predicted == labels).sum().item()

    val_acc_tune = 100. * correct_val_tune / total_val_tune
    lr_results[lr_val] = val_acc_tune
    print(f"    LR={lr_val} -> Validation Accuracy after
{num_epochs_tuning} epochs: {val_acc_tune:.2f}%")
```

```
Trying Hyperparameter Tuning (Learning Rate)...

  Tuning with Learning Rate: 0.0005
```


```
{"model_id":"28a576ec63c54125be1d47a79f1a2777","version_major":2,"vers
ion_minor":0}

{"model_id":"8fc8bf2aee6a42678bface77b4c1182a","version_major":2,"vers
ion_minor":0}

{"model_id":"40e2fdaea3e34873b8898b38696da7d2","version_major":2,"vers
ion_minor":0}

{"model_id":"5f61a09eded7486bb9c8c68d90745183","version_major":2,"vers
ion_minor":0}

{"model_id":"4c0e683bfc254da6b38025410d9340dd","version_major":2,"vers
ion_minor":0}
```


```
    Finished 5 epochs for LR=0.0005
    LR=0.0005 -> Validation Accuracy after 5 epochs: 74.99%

  Tuning with Learning Rate: 0.001
```


```
{"model_id":"37cccf467ea14512bdb30af819ff4717","version_major":2,"vers
ion_minor":0}
```

{"model_id":"e81995d1cf3944c4a595da45ed4511cf","version_major":2,"version_minor":0}

{"model_id":"e88782956fb44e87910780ccf3c19aba","version_major":2,"version_minor":0}

{"model_id":"65c4c4154bcb40b08a692dad330881c3","version_major":2,"version_minor":0}

{"model_id":"e214713b36de493eba63d89aada97cea","version_major":2,"version_minor":0}

```
    Finished 5 epochs for LR=0.001
    LR=0.001 -> Validation Accuracy after 5 epochs: 74.34%

  Tuning with Learning Rate: 0.005
```

{"model_id":"c27d3caf272c48339d9dd3a7f88137ca","version_major":2,"version_minor":0}

{"model_id":"b3aa74c990ae4858a9da5bbe4d793787","version_major":2,"version_minor":0}

{"model_id":"bec1c04a91264014ae829ec04e6aed27","version_major":2,"version_minor":0}

{"model_id":"f076a93062f9498b92a7c9119f98d5f8","version_major":2,"version_minor":0}

{"model_id":"22041bc072ea42c4b15da64d8d963599","version_major":2,"version_minor":0}

```
    Finished 5 epochs for LR=0.005
    LR=0.005 -> Validation Accuracy after 5 epochs: 53.11%
```

```
print("\nHyperparameter Tuning Results (Learning Rate vs. Validation Accuracy):")
for lr, acc in lr_results.items():
    print(f"  Learning Rate: {lr} -> Validation Accuracy: {acc:.2f}%")

best_lr = max(lr_results, key=lr_results.get)
print(f"Best performing learning rate from this short test: {best_lr} with accuracy {lr_results[best_lr]:.2f}%")
```

```
Hyperparameter Tuning Results (Learning Rate vs. Validation Accuracy):
  Learning Rate: 0.0005 -> Validation Accuracy: 74.99%
  Learning Rate: 0.001 -> Validation Accuracy: 74.34%
  Learning Rate: 0.005 -> Validation Accuracy: 53.11%
Best performing learning rate from this short test: 0.0005 with accuracy 74.99%
```